@Suryakant
Founder: NxtHiring

@heyitsurya

Artificial Intelligence

Complete Notes

Machine Learning · Deep Learning · NLP · Generative AI

BEGINNER TO ADVANCED

- ☑ AI vs ML vs DL & the maths behind it
- ☑ Regression, classification & clustering
- ☑ Evaluation, tuning & ensembles
- ☑ Neural networks, CNNs & Transformers
- ☑ LLMs, prompting, RAG & AI agents
- ☑ MLOps, ethics + Interview Q&A

SWIPE TO NEXT →

@Suryakant

Table of Contents ALL TOPICS

16 pages from "what is AI?" to building LLM apps — colour-coded by topic.

AI Basics | Machine Learning | Deep Learning | NLP | Generative AI | MLOps & Ethics

03 What is AI? AI vs ML vs DL vs GenAI ..... AI BASICS

04 Math & Data Foundations ..... AI BASICS

05 Machine Learning Basics & Workflow ..... MACHINE LEARNING

06 Regression: Predicting Numbers ..... MACHINE LEARNING

07 Classification & Metrics ..... MACHINE LEARNING

08 Unsupervised Learning: Clustering & PCA ..... MACHINE LEARNING

09 Model Evaluation, Overfitting & Tuning ..... MACHINE LEARNING

10 Decision Trees, Ensembles & Feature Engineering ..... MACHINE LEARNING

11 Neural Networks & Deep Learning ..... DEEP LEARNING

12 CNNs & Computer Vision ..... DEEP LEARNING

13 NLP, Attention & Transformers ..... NLP

14 Large Language Models & Generative AI ..... GENERATIVE AI

15 Prompt Engineering, RAG & AI Agents ..... GENERATIVE AI

16 MLOps, Deployment & Responsible AI ..... MLOPS & ETHICS

17 Interview Questions & Answers ..... ALL TOPICS

18 Cheat Sheet & Summary ..... ALL TOPICS

Important Note

Study in order: maths → classic ML → deep learning → transformers → LLM apps. Code every example in a Jupyter notebook.

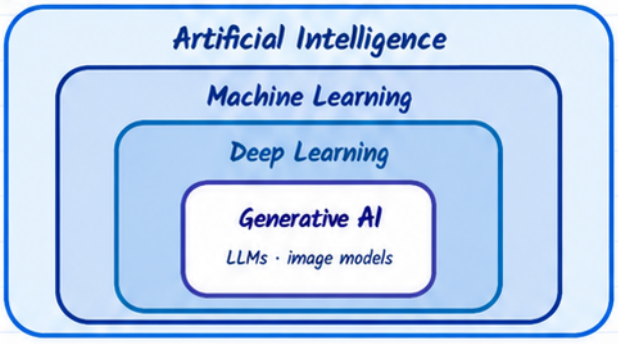
★ Key Takeaway

Most AI jobs still need strong ML fundamentals — GenAI sits ***on top*** of them, not instead of them.

What is AI? AI BASICS

Artificial Intelligence = machines doing tasks that normally need human intelligence: learning, reasoning, seeing and understanding language.

① AI inside AI: the nested view



② Simple definitions

- **AI** – any technique that makes machines act intelligently (rules or learning)
- **ML** – systems that learn patterns from data instead of hand-written rules
- **DL** – ML using many-layered neural networks
- **GenAI** – DL models that *create* new text, images, audio or code

③ Traditional programming vs Machine Learning

Traditional: Rules + Data → **Answers**
IF amount > $5,000 AND country ≠ home → flag fraud

ML: Data + Answers → **Rules (model)**
Learn fraud patterns from 1M labelled past transactions

④ Key milestones

Year	Milestone
1950	Alan Turing proposes the "imitation game" (Turing Test)
1956	Dartmouth workshop – the term "Artificial Intelligence" is born
1997	IBM Deep Blue beats chess champion Garry Kasparov
2012	AlexNet wins ImageNet → deep learning boom
2016	DeepMind AlphaGo beats Go champion Lee Sedol
2017	"Attention Is All You Need" introduces the Transformer
2022	ChatGPT Launches (Nov) → Generative AI goes mainstream

⑤ Types by capability

Type	Meaning
Narrow AI (ANI)	Does specific tasks – **all AI today**
General AI (AGI)	Human-level across any task – not yet achieved
Super AI (ASI)	Beyond human ability – hypothetical

⑥ AI in business

Industry	Use case
Banking	Fraud detection, credit scoring
Retail	Recommendations, demand forecast
Healthcare	Medical-image screening
Support	Chatbots, ticket routing

★ Key Takeaway

Every DL model is ML and every ML model is AI – but not the other way round.

⚠ Common Mistakes

× Calling any automation or IF-rule "AI"
× Thinking ChatGPT = all of AI (it is one GenAI product)

Math & Data Foundations AI BASICS

Four pillars behind every AI model: linear algebra, statistics, probability and calculus.

① Linear algebra — data as numbers

Object	Example	Shape
Scalar	5	()
Vector	[1, 2, 3]	(3,)
Matrix	a table / image	(rows, cols)
Tensor	batch of colour images	(32, 224, 224, 3)

Dot product: [1,2,3]·[4,5,6] = 4+10+18 = **32**

Matrix multiply: (m×n).(n×p) → (m×p) — inner sizes must match.

② Statistics — describe data

Data: 2, 4, 4, 4, 5, 5, 7, 9

Measure	Value
Mean	40 ÷ 8 = **5**
Median	(4+5) ÷ 2 = **4.5**
Mode	**4**
Variance (population)	**4**
Std deviation	√4 = **2**

Normal curve: 68% / 95% / 99.7% within 1 / 2 / 3 std.

③ Probability & Bayes' theorem

P(A|B) = P(B|A) × P(A) ÷ P(B)

Disease affects 1%. Test catches 90% of sick people, but gives 5% false positives. You test positive — chance you are sick?

P = (0.90 x 0.01) ÷ (0.90 x 0.01 + 0.05 x 0.99) = 0.009 ÷ 0.0585 = **15.4%** — not 90%! (Naive Bayes classifiers use this rule.)

④ Calculus — how models learn

- Derivative = slope: how loss changes if a weight changes
- Gradient = vector of all partial derivatives
- **Gradient descent:** w ← w − η · ∂Loss/∂w (η = learning rate)

```
# Loss = (w - 3)^2  ->  dLoss/dw = 2(w - 3)
# start w = 0, learning rate 0.1
step 1: w = 0   - 0.1*(-6.0) = 0.60
step 2: w = 0.6 - 0.1*(-4.8) = 1.08
... w moves toward 3 (minimum)
```

⑤ Data types AI works with

Type	Examples
Structured	Tables: sales, customers
Unstructured	Text, images, audio, video
Semi-structured	JSON, XML, logs
Time series	Daily sales, sensor data

```
import numpy as np
np.dot([1,2,3], [4,5,6])   # 32
np.std([2,4,4,4,5,5,7,9])  # 2.0
```

★ Key Takeaway

You don't need a maths degree — know ***what each concept does*** to a model: vectors store data, gradients train it, statistics judge it.

Interview Tip

"Why does the learning rate matter?" → too big overshoots and diverges, too small trains very slowly.

Machine Learning Basics MACHINE LEARNING

Machine Learning = giving computers data and answers so they learn the rules themselves.

1 Three ways machines learn

Type	Data	Goal	Business example
Supervised	Labelled (X → y)	Predict a label / number	Spam or not, house price
Unsupervised	No labels	Find structure	Customer segments
Reinforcement	Rewards from actions	Best strategy	Game AI, robots, ad bidding
Self-supervised	Labels made from data itself	Learn representations	LLM predicts the next word

2 The ML project workflow

Define problem → **Collect data** → **Clean & EDA** → **Feature engineering** → **Split data** → **Train** → **Evaluate** → **Tune** → **Deploy** → **Monitor**

3 Key vocabulary

Term	Meaning
Feature (X)	Input column, e.g. tenure
Label / target (y)	What we predict, e.g. churn
Model	Learned function f(X) ≈ y
Training	Fitting the model to data
Inference	Predicting on new data
Parameter	Learned (weights)
Hyperparameter	Set by you (max_depth, k)

4 Train / test split

Train on one part, test on ***unseen*** data (typically 80/20 or 70/15/15 with validation).

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    random_state=42,    # reproducible
    stratify=y)         # keep class ratio
```

5 The scikit-learn pattern

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X_train, y_train)        # learn
preds = model.predict(X_test)      # infer
model.score(X_test, y_test)        # accuracy
```

6 Pick an algorithm

Goal	Start with
Predict a number	Linear Regression → Random Forest
Predict a category	Logistic Regression → XGBoost
Find groups	K-Means
Reduce columns	PCA
Images / text	Deep Learning

⚠ Common Mistakes

- ✕ **Data leakage**: scaling or filling NaNs on all data before splitting
- ✕ Judging a model on its own training data

💡 Interview Tip

Always mention a ***baseline*** (e.g. predict the average) — a model is only useful if it beats it.

Regression MACHINE LEARNING

Regression predicts continuous values — sales, prices, demand — by fitting the best line (or curve) through data.

① Simple linear regression

$y = b_0 + b_1x$ (b_0 intercept, b_1 slope)

Ad spend ($k)	1	2	3	4	5
Sales ($k)	14	18	23	25	30
Predicted	14.2	18.1	22.0	25.9	29.8

Best-fit line: **Sales = 10.3 + 3.9 × Spend**

Each extra $1k ad spend → about $3.9k more sales. Spend $6k → 33.7k.

② Error metrics (same data)

Metric	Formula	Value
MAE	mean |y − ŷ|	0.48
MSE	mean $(y - \hat{y})^2$	0.38
RMSE	√MSE (same units as y)	0.62
R^2	1 − SSres ÷ SStot	0.988

R^2 = 0.988 → the line explains 98.8% of the variation in sales. RMSE punishes big errors more than MAE.

③ Code it in scikit-learn

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

X = [[1], [2], [3], [4], [5]]          # 2-D!
y = [14, 18, 23, 25, 30]
lr = LinearRegression().fit(X, y)
lr.intercept_, lr.coef_      # 10.3, [3.9]
lr.predict([[6]])            # [33.7]
```

④ How the line is found

- **Cost function:** minimise MSE (sum of squared errors)
- **OLS** solves it exactly; big data uses gradient descent
- **Multiple regression:** $y = b_0 + b_1x_1 + b_2x_2 + \ldots$ — each b is the effect holding others constant

⑤ Other regression models

Model	Use when
Polynomial	Curved relationship
Ridge (L2)	Many correlated features
Lasso (L1)	Want automatic feature selection
Random Forest / XGBoost	Non-linear, tabular data

⑥ Linear regression assumptions

- ☑ Linear relationship
- ☑ Independent errors
- ☑ Constant error variance (homoscedasticity)
- ☑ Errors roughly normal
- ☑ No strong multicollinearity (check VIF)

★ Key Takeaway

Regression predicts a **number**. Report RMSE/MAE in business units ("±$620 per forecast"), not only R^2.

⚠ Common Mistakes

- ✕ Passing a 1-D list as X (sklearn needs 2-D)
- ✕ Reading a high R^2 as proof of causation

Classification & Metrics

MACHINE LEARNING

Classification predicts a category — fraud / not fraud, churn / stay — and is judged with the confusion matrix.

1 Logistic regression

$\sigma(z) = 1 \div (1 + e^{-z})$ turns any score into a probability 0–1

- $\sigma(0) = 0.5$ · $\sigma(2) = 0.88$ · $\sigma(-1) = 0.27$
- Default rule: probability ≥ 0.5 → class 1
- Despite the name, it is a ***classifier***

2 Popular classifiers

Model	Idea
Logistic Regression	Linear boundary + sigmoid
K-Nearest Neighbors	Vote of the k closest points
Decision Tree	Series of yes/no splits
Naive Bayes	Bayes' rule, fast for text
SVM	Widest-margin boundary
Random Forest / XGBoost	Many trees combined

3 Confusion matrix — 1,000 transactions

	Predicted Fraud	Predicted OK
Actual Fraud	**TP = 40**	**FN = 10**
Actual OK	**FP = 20**	**TN = 930**

50 real frauds, 950 normal.

4 Metrics from it

Metric	Formula	Value
Accuracy	(TP+TN) ÷ all	97.0%
Precision	TP ÷ (TP+FP)	66.7%
Recall	TP ÷ (TP+FN)	80.0%
F1	2PR ÷ (P+R)	72.7%
Specificity	TN ÷ (TN+FP)	97.9%

5 Precision or recall?

Case	Prioritise
Spam filter (don't lose real email)	Precision
Cancer / fraud screening (don't miss cases)	Recall
Need balance	F1-score

ROC-AUC: 0.5 = random guess, 1.0 = perfect ranking.

6 Code

```
from sklearn.linear_model import
LogisticRegression
from sklearn.metrics import classification_report

clf = LogisticRegression(class_weight="balanced")
clf.fit(X_train, y_train)
proba = clf.predict_proba(X_test)[:, 1]
pred = (proba >= 0.3).astype(int)  # tune
threshold
print(classification_report(y_test, pred))
```

⚠ Common Mistakes

- ✕ Trusting **accuracy** on imbalanced data — predicting "OK" always scores 95% here
- ✕ Forgetting to tune the decision threshold

Interview Tip

Explain FP vs FN in money: "a missed fraud costs $2,000, a false alarm costs a $5 phone call".

Unsupervised Learning

MACHINE LEARNING

No labels — the algorithm finds hidden structure: groups, compressed features, unusual points and items bought together.

1 K-Means clustering

Choose k → **Place k centroids** → **Assign points to nearest** → **Move centroids to mean** → **Repeat until stable**

- **Elbow method**: plot inertia vs k, pick the bend
- **Silhouette score**: -1 to 1, higher = better-separated clusters
- Scale features first — K-Means uses distance

2 Code

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

Xs = StandardScaler().fit_transform(rfm)
km = KMeans(n_clusters=3, n_init=10,
            random_state=42)
rfm["segment"] = km.fit_predict(Xs)
```

3 Example result — customer segments (RFM)

Segment	Recency (days)	Frequency	Monetary	Action
Champions	12	18	$4,200	Loyalty rewards, early access
At risk	140	6	$1,100	Win-back discount
New / low	25	1	$90	Welcome series

4 Other clustering

Method	Strength
Hierarchical	Dendrogram, no k upfront
DBSCAN	Any shape, flags outliers as noise
Gaussian Mixture	Soft (probability) clusters

5 PCA — dimensionality reduction

- Creates new uncorrelated axes (components) that keep the most variance
- Use for visualisation (2-D plot), speed and removing noise
- Check *explained_variance_ratio_* — e.g. keep 95%

6 Association rules (market basket)

1,000 baskets: bread 200, butter 150, both 100

Measure	Calculation	Value
Support	100 ÷ 1,000	10%
Confidence (bread → butter)	100 ÷ 200	50%
Lift	0.50 ÷ 0.15	3.33

Lift > 1 → bought together more than by chance.

7 Anomaly detection

- Z-score: flag $|z| > 3$
- IQR rule: below Q1 − 1.5·IQR or above Q3 + 1.5·IQR
- Isolation Forest for many features
- Uses: fraud, faulty sensors, network attacks

★ Key Takeaway

Unsupervised learning has no "right answer" — clusters must be ***named and validated*** with business sense.

Important Note

K-Means assumes round, similar-sized clusters. For odd shapes or outliers, try DBSCAN.

Evaluation & Tuning

MACHINE LEARNING

A model is only good if it works on data it has never seen. Measure honestly, then tune.

① Underfitting vs overfitting

	Underfit	Good fit	Overfit
Train error	High	Low	Very low
Test error	High	Low	High
Problem	High bias	Balanced	High variance
Fix	More features, complex model	—	More data, regularise, simplify

② Bias-variance trade-off

③ K-fold cross-validation

Split data into k = 5 folds; train on 4, test on 1; repeat 5 times; average the scores → a more reliable estimate.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y,
                         cv=5, scoring="f1")
scores.mean(), scores.std()
```

④ Regularisation

Technique	Effect
L1 / Lasso	Pushes some weights to 0 (feature selection)
L2 / Ridge	Shrinks all weights
Dropout	Randomly drops neurons (NN)
Early stopping	Stop when validation loss rises
Pruning / max_depth	Smaller trees

⑤ Hyperparameter tuning

```
from sklearn.model_selection import GridSearchCV

grid = {"n_estimators": [100, 300],
        "max_depth": [5, 10, None]}
gs = GridSearchCV(RandomForestClassifier(),
                  grid, cv=5, scoring="roc_auc")
gs.fit(X_train, y_train)
gs.best_params_, gs.best_score_
```

Faster: RandomizedSearchCV or Bayesian search (Optuna).

⑥ Avoid data leakage with a Pipeline

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
   ("scale", StandardScaler()),
   ("model", LogisticRegression())])
# scaler is fit on training folds only
cross_val_score(pipe, X, y, cv=5)
```

- Time series: split by date, never shuffle (TimeSeriesSplit)
- Test set is touched **once**, at the very end

★ Key Takeaway

Train well, ***validate*** to choose, ***test*** once to report. Big gap between train and test score = overfitting.

Interview Tip

"How do you know a model is overfitting?" → training score far above validation score: fix with more data, regularisation or simpler model.

Trees, Ensembles & Features

MACHINE LEARNING

Combine many weak trees into one strong model, and shape raw columns into features a model can learn from.

① Decision trees

- Split data with yes/no questions: *tenure < 6 months?*
- Choose the split that makes groups purest
- **Gini** = $1 - \Sigma p^2$. Node with 8 churn / 2 stay: $1 - (0.8^2 + 0.2^2)$ = **0.32**
- **Entropy** = $-\Sigma p \cdot \log_2 p$ (0 = pure)
- Easy to explain, but one deep tree overfits

② Bagging vs boosting

	Bagging	Boosting
Example	Random Forest	XGBoost, LightGBM, CatBoost
Trees built	In parallel, random samples	One after another, fixing errors
Mainly reduces	Variance	Bias
Overfit risk	Low	Higher — tune learning rate

③ Feature engineering toolkit

Task	Technique	Example
Missing values	Impute median / mode, add "was_missing" flag	income NaN → median $52k
Categories	One-hot (nominal), ordinal encoding (ordered)	city → city_Lahore 0/1; size S<M<L → 0,1,2
Scaling	StandardScaler $z = (x - \mu) \div \sigma$; MinMax → [0, 1]	needed for KNN, SVM, NN, K-Means
Dates	Extract month, weekday, is_weekend, days_since	order_date → weekday = Fri
Skewed values	$\log(1 + x)$	revenue 10 … 1,000,000
New ratios	Combine columns	avg_order = revenue ÷ orders

④ Full pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

prep = ColumnTransformer([
   ("num", StandardScaler(), ["tenure", "spend"]),
   ("cat", OneHotEncoder(handle_unknown="ignore"),
         ["plan", "city"])])
pipe = Pipeline([("prep", prep),
   ("rf", RandomForestClassifier(n_estimators=300))])
pipe.fit(X_train, y_train)
```

⑤ Explain the model

- *feature_importances_* — quick tree ranking
- Permutation importance — drop in score when a column is shuffled
- **SHAP** — how much each feature pushed *one* prediction up or down
- Partial dependence plots

★ Key Takeaway

For tabular business data, **gradient-boosted trees** are usually the strongest starting point — and good features beat fancy models.

Interview Tip

"Why don't trees need scaling?" → splits are thresholds on one feature, so rescaling doesn't change which split is best.

Neural Networks & Deep Learning

DEEP LEARNING

Neural networks stack layers of simple "neurons" that learn their own features from raw data.

1 One neuron

output = activation(w · x + b)

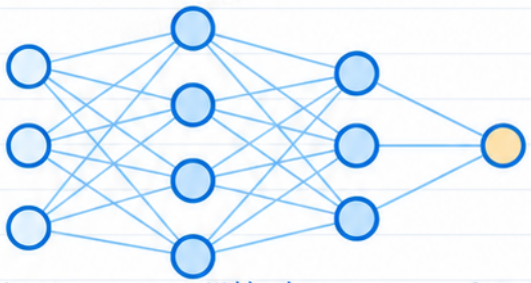
x = [2, 3], w = [0.5, −1], b = 1

z = 0.5·2 + (−1)·3 + 1 = **−1**

ReLU(−1) = **0** · sigmoid(−1) = **0.27**

- **Weights** = how important each input is
- **Bias** = shifts the output up or down
- **Activation** = adds non-linearity

2 A network = layers of neurons



3 Activation functions

Function	Output	Use
ReLU max(0, x)	0 → ∞	Hidden layers (default)
Sigmoid	0 → 1	Binary output
Softmax	Probabilities sum to 1	Multi-class output
Tanh	−1 → 1	RNN hidden states
Linear	Any number	Regression output

4 Loss functions

Task	Loss
Regression	MSE / MAE
Binary class	Binary cross-entropy
Multi-class	Categorical cross-entropy

Without non-linear activations, many layers collapse into one linear model.

5 How a network learns

Forward pass → prediction → **Compute loss** → **Backpropagation (chain rule → gradients)** → **Optimizer updates weights (SGD / Adam)** → **Repeat for many epochs**

Epoch = one pass over all data · **Batch size** = samples per update · **Learning rate** = step size

6 Keras example (binary classifier)

```
from tensorflow import keras

model = keras.Sequential([
    keras.Input(shape=(20,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(1, activation="sigmoid")])
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.fit(X_train, y_train, epochs=20,
          batch_size=32, validation_split=0.2)
```

7 Training problems & fixes

Problem	Fix
Vanishing gradients	ReLU, batch norm, residual (skip) connections
Exploding gradients	Gradient clipping
Overfitting	Dropout, early stopping, more data
Slow training	GPU, Adam, smaller model

Frameworks: **PyTorch**, TensorFlow/Keras, JAX.

★ Key Takeaway

Deep learning shines on ***unstructured data*** (images, text, audio); for small tables, tree models often win.

⚠ Common Mistakes

- ✕ Feeding unscaled inputs to a neural net
- ✕ Using sigmoid in every hidden layer → vanishing gradients

CNNs & Computer Vision

DEEP LEARNING

Convolutional Neural Networks learn visual patterns — edges → shapes → objects — and power most computer vision.

① Images are tensors

- Height × width × channels (RGB = 3)
- 224 × 224 × 3 = **150,528** numbers per image
- Pixel values 0–255 → scale to 0–1

② Convolution in 1 line

A small filter (e.g. 3×3) slides over the image and multiplies-and-adds → a ***feature map***. Early filters find edges, later ones find shapes and objects.

③ Size formulas

Output = (W − F + 2P) ÷ S + 1

W input, F filter, P padding, S stride

Input	F	P	S	Output
32	3	1	1	32 ("same")
28	5	0	1	24
24	2 (max-pool)	0	2	12

Max-pool 2×2 of [[1, 3], [2, 8]] → **8**

④ Typical CNN architecture

Image → Conv + ReLU → Pooling → Conv + ReLU → Pooling → Flatten → Dense → Softmax → class

⑤ Famous architectures

Model	Year	Big idea
LeNet-5	1998	Digit recognition
AlexNet	2012	GPUs + ReLU, won ImageNet
VGG-16	2014	Deep stacks of 3×3 convs
ResNet	2015	Skip connections, 152 layers
YOLO	2016	Real-time object detection
ViT	2020	Transformer on image patches

⑥ Computer-vision tasks

Task	Answers	Example
Classification	What is it?	Defective / OK part
Detection	What + where (boxes)	Count cars
Segmentation	Which pixels?	Tumour outline
OCR	What text?	Read invoices
Face recognition	Who is it?	Phone unlock

⑦ Transfer learning (best practice)

```
from keras.applications.ResNet50(
    weights="imagenet", include_top=False,
    input_shape=(224, 224, 3), pooling="avg")
base.trainable = False          # freeze
model = keras.Sequential([base,
    keras.layers.Dense(S, activation="softmax")])
model.fit(X_train, y_train)
```

⑧ Data augmentation

- ☑ Random flips & rotations
- ☑ Crops & zoom
- ☑ Brightness / contrast changes
- ☑ Creates variety → less overfitting

Apply to training images only.

★ Key Takeaway

Don't train from scratch: start from a ***pre-trained*** model and fine-tune → you need far fewer images.

Interview Tip

"Why CNN instead of a dense network for images?" → shared filters = far fewer weights, and they detect a pattern anywhere in the image.

NLP & Transformers NLP

Natural Language Processing lets machines read, understand and generate human language.

1 NLP pipeline

Raw text → Clean (lowercase, remove noise) → Tokenize → Vectorize / embed → Model → Output (label, answer, summary)

2 Turning words into numbers

Method	Idea
Bag of Words	Count of each word; order lost
TF-IDF	tf × log(N ÷ df): rare, useful words score high
Word2Vec / GloVe	Dense vectors; king − man + woman ≈ queen
Contextual (BERT, GPT)	Vector depends on the sentence

3 Sequence models

Model	Note
RNN	Reads word by word; forgets long context
LSTM / GRU	Gates keep long-term memory
Transformer	Attention over all words at once; parallel, scales

4 Attention — the core idea

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T \div \sqrt{d_k}) \cdot V$$

Each token builds a **Query**, compares it with every token's **Key**, and takes a weighted mix of their **Values**. So "bank" in "*river bank*" vs "*bank loan*" gets different meanings from its neighbours. Multi-head attention does this several ways in parallel.

5 Transformer families

Type	Models	Best for
Encoder-only	BERT, RoBERTa	Classification, NER, search
Decoder-only	GPT, Llama, Mistral	Text generation, chat
Encoder-decoder	T5, BART	Translation, summarisation

6 Common NLP tasks

- ☑ Sentiment analysis
- ☑ Named-entity recognition (NER)
- ☑ Text classification / topic
- ☑ Translation & summarisation
- ☑ Question answering
- ☑ Semantic search

7 Hugging Face in 3 lines

```
from transformers import pipeline

clf = pipeline("sentiment-analysis")
clf("Fast delivery and great support!")
# [{'label': 'POSITIVE', 'score': 0.99...}]
```

8 Tokenization

- Text split into **tokens** (words or sub-words)
- "unhappiness" → "un" + "happiness"
- Methods: BPE, WordPiece, SentencePiece
- Models have a max token length

★ Key Takeaway

The 2017 Transformer replaced recurrence with **attention** — the foundation of BERT, GPT and every modern LLM.

⚠ Common Mistakes

- ✕ Removing stop-words like "not" before sentiment analysis
- ✕ Using a tokenizer that doesn't match the model

LLMs & Generative AI

GENERATIVE AI

Generative AI creates new content. Large Language Models (LLMs) are its text engine.

1 What is an LLM?

- A large decoder Transformer with billions of parameters
- Trained to ***predict the next token*** on huge amounts of text
- Emergent skills: writing, coding, reasoning, translation
- Output is probabilistic — sampled token by token

2 Key terms

Term	Meaning
Token	≈ 4 characters ≈ ¾ of an English word
Context window	Max tokens it can read + write at once
Temperature	Low = focused, high = more random
Top-p	Sample only from the most likely tokens
Hallucination	Confident but false output
Embedding	Vector meaning of text

3 How LLMs are built

Pre-training (next-token on web-scale text) → ***Supervised fine-tuning (instruction examples)*** → ***Preference tuning (RLHF / DPO)*** → ***Safety testing*** → ***Deploy via app / API***

4 Generative model families

Family	How it works	Examples
LLMs (Transformers)	Predict next token	GPT, Claude, Gemini, Llama
Diffusion	Learn to remove noise step by step	Stable Diffusion, DALL-E 3
GANs (2014)	Generator vs discriminator game	Face / style generation
VAEs	Encode → sample → decode	Image variation
Multimodal	Text + image + audio in one model	Image Q&A, voice assistants

5 Customising an LLM — cheapest first

Step	When
1. Prompt engineering	Always start here
2. RAG	Needs your private / fresh data
3. Fine-tuning (LoRA)	Specific style, format or task
4. Pre-train	Rarely — very expensive

6 Call an LLM from Python

```
import anthropic

client = anthropic.Anthropic()  # API key from env
msg = client.messages.create(
    model="<model-name>", max_tokens=300,
    messages=[{"role": "user",
        "content": "Summarise Q2 sales in 3 bullets: ..."}]
)
print(msg.content[0].text)
```

7 Limitations

- Hallucinations — verify facts
- Knowledge cutoff date
- Bias from training data
- Prompt injection attacks
- Cost & latency per token

★ Key Takeaway

An LLM is a ***next-token predictor*** trained at huge scale — great at language, but it needs grounding and checks for facts.

Interview Tip

How would you reduce hallucinations? → RAG with sources, low temperature, ask for citations, and evaluate on a test set.

Prompting, RAG & Agents GENERATIVE AI

How to get reliable work out of LLMs: write better prompts, ground them in your data, and let them use tools.

1 Prompting techniques

Technique	How
Zero-shot	Just the instruction
Few-shot	Give 2–5 worked examples
Chain-of-thought	Ask it to reason step by step
Role / system prompt	"You are a senior data analyst..."
Structured output	Ask for JSON with named fields
Delimiters	Wrap data in <data>...</data>

2 A strong prompt

Role + Task + Context + Format + Constraints

```
You are a retail data analyst.
Task: explain why South revenue fell in Q2.
Context: <data> ...monthly table... </data>
Format: 3 bullets + 1 recommendation.
Rules: use only the data; say
"not enough data" if unsure.
```

3 RAG — Retrieval-Augmented Generation

Documents → **Split into chunks** → **Embed (vectors)** → **Store in vector DB**

User question → **Embed question** → **Retrieve top-k similar chunks** → **Add chunks to prompt** → **LLM answers with sources**

Similarity = **cosine**: $A \cdot B \div (|A| \cdot |B|)$, 1 = same direction. Vector stores: FAISS, Chroma, Pinecone, **pgvector**.

4 RAG vs fine-tuning

Need	Use
Company docs, fresh data	RAG
Cite sources	RAG
Consistent tone / format	Fine-tuning
New narrow skill	Fine-tuning

5 AI agents

Agent = LLM + tools + memory + a loop

Goal → **Plan** → **Act (call tool)** → **Observe result** → **Repeat / finish**

- Tools: web search, SQL, Python, email, APIs
- Frameworks: LangChain / LangGraph, LlamaIndex, CrewAI
- MCP (Model Context Protocol) connects tools to models

6 Evaluate LLM apps

Check	Question
Groundedness	Is every claim supported by retrieved text?
Relevance	Did retrieval find the right chunks?
Accuracy	Correct on a labelled test set of questions?
Safety	Resists prompt injection, no data leaks?

★ Key Takeaway

Start with a clear prompt, add **RAG** for knowledge, add **tools** for actions — and measure every change.

⚠ Common Mistakes

- ✕ Chunks too big or too small for retrieval
- ✕ Giving an agent tools with no permission limits or human approval

MLOps & Responsible AI

MLOPS & ETHICS

Take models from notebook to production — and make sure they stay accurate, fair and safe.

1 MLOps lifecycle

Data & versioning → Train → Track experiments (MLflow) → Package (Docker) → Deploy (API) → Monitor → Retrain

2 Serve a model as an API

```
import joblib
from fastapi import FastAPI

app = FastAPI()
model = joblib.load("churn_model.pkl")

@app.post("/predict")
def predict(features: list[float]):
    p = model.predict_proba([features])[0, 1]
    return {"churn_probability": round(float(p), 3)}
```

3 Deployment options

Mode	Example
Batch	Score all customers every night
Real-time API	Fraud check in < 100 ms
Streaming	Sensor alerts from Kafka
Edge / on-device	Phone face unlock

4 Monitoring

Issue	Meaning	Example
Data drift	Input distribution changes	New customer age mix
Concept drift	X → y relationship changes	Fraudsters change tactics
Performance decay	Accuracy drops over time	Recall 80% → 62%

5 MLOps tools

- ☑ Git + DVC (data versions)
- ☑ MLflow / W&B (experiments)
- ☑ Docker + Kubernetes
- ☑ Cloud: AWS SageMaker, Azure ML, Vertex AI
- ☑ Evidently (drift reports)

6 Responsible AI principles

Principle	In practice
Fairness	Compare error rates across groups
Transparency	Explain decisions (SHAP, LIME)
Privacy	Minimise & anonymise personal data (GDPR)
Safety & security	Red-team, guard against misuse
Accountability	Human review for high-stakes decisions

7 Regulation & real cases

- ***EU AI Act*** (in force Aug 2024): risk tiers — unacceptable (banned), high, limited (transparency), minimal
- Amazon scrapped an AI hiring tool (reported 2018) that downgraded women's CVs — biased training data
- Document data sources, intended use and limits (model cards)

★ Key Takeaway

A model in a notebook creates no value — ***deployed, monitored and fair*** models do.

Interview Tip

"How do you monitor a model in production?" → track data & concept drift, set alerts, and retrain when performance drops.

Interview Q & A ALL TOPICS

12 AI / ML questions asked again and again in interviews — with short, correct answers.

Q1 AI vs ML vs Deep Learning? AI basics

A: AI is the broad goal of intelligent machines; ML is AI that learns from data; DL is ML using multi-layer neural networks.

Q2 Supervised vs unsupervised learning? ML

A: Supervised learns from labelled examples to predict y (regression, classification); unsupervised finds structure in unlabelled data (clustering, PCA).

Q3 What is overfitting and how do you prevent it? ML

A: The model memorises training noise and fails on new data. Prevent with more data, cross-validation, regularisation, simpler models, early stopping.

Q4 Explain the bias-variance trade-off. ML

A: Simple models underfit (high bias); complex models overfit (high variance). Aim for the complexity with the lowest validation error.

Q5 Precision vs recall? ML

A: Precision = how many predicted positives are right; recall = how many actual positives were found. Fraud/medical favour recall, spam favours precision.

Q6 Bagging vs boosting? ML

A: Bagging trains trees in parallel on random samples and averages (Random Forest, lowers variance); boosting trains sequentially to fix errors (XGBoost, lowers bias).

Q7 How do you handle imbalanced data? ML

A: Use precision/recall/F1 or PR-AUC instead of accuracy; class weights, resampling (SMOTE), and tune the decision threshold.

Q8 Why do neural networks need activation functions? DL

A: They add non-linearity; without them any number of layers equals one linear model.

Q9 What is gradient descent? DL

A: An optimiser that updates weights in the opposite direction of the loss gradient, step size set by the learning rate.

Q10 What is attention in a Transformer? NLP

A: Each token weighs every other token (softmax of Q·K scores) to build context-aware representations, processed in parallel.

Q11 What is RAG and why use it? GenAI

A: Retrieve relevant document chunks from a vector store and add them to the prompt, so the LLM answers from current, private data with sources.

Q12 What is hallucination and how to reduce it? GenAI

A: Fluent but false output. Reduce with RAG/grounding, lower temperature, asking for citations, and evaluation with human review.

Interview Tip

Answer in 3 parts: definition → example from your project → why it matters to the business.

Common Mistakes

- ✕ Reciting formulas without saying when to use them
- ✕ Claiming an LLM "knows" facts → it predicts tokens

Cheat Sheet & Summary

ALL TOPICS

All of AI on one page — screenshot it and keep it next to you while you practise.

1 AI project flow

Business problem → Data → Features → Model → Evaluate → Deploy → Monitor

Machine Learning

- train_test_split(stratify=y)
- Pipeline(scaler → model)
- Regression: RMSE · MAE · R^2
- Classification: precision · recall · F1 · AUC
- K-Means + elbow / silhouette
- RandomForest · XGBoost for tables
- cross_val_score · GridSearchCV

Deep Learning

- neuron = act(w·x + b)
- ReLU hidden · sigmoid / softmax out
- Loss → backprop → Adam
- Dropout · early stopping
- CNN: conv → pool → dense
- (W − F + 2P) ÷ S + 1
- Transfer learning (ResNet, ViT)

NLP & Transformers

- Tokenize → embed → model
- TF-IDF · Word2Vec · BERT
- $\text{softmax}(QK^T \div \sqrt{d_k}) \cdot V$
- Encoder = understand (BERT)
- Decoder = generate (GPT)
- pipeline("sentiment-analysis")
- Watch max token length

Generative AI

- LLM = next-token predictor
- Pre-train → SFT → RLHF
- Prompt: role · task · context · format
- RAG: chunk → embed → retrieve
- Agent: plan → act → observe
- Temperature low for facts
- Evaluate · guard · human review

★ Key Takeaway

Master the fundamentals (data, ML, evaluation) — every new AI tool builds on them.

Interview Tip

Build 3 projects: a churn predictor, an image classifier and a RAG chatbot over your own PDFs.

Follow @Suryakant for more AI Insights

Save this post • Share with a friend

Turn raw data into actionable insights